

FAISS 벡터DB 파일 검색 테스트

도로교통법 PDF를 벡터DB에 저장하고 자연어 질문으로 검색하기

작성일: 2026년 10월 6일

작성자: 심성욱

코드: github.com/SKN35shimsungwook/faiss-vectordb-file-search-test

[목차]

1. 활용 데이터
2. 프로세스 설계
3. 구현
4. 결과 및 한계점

1. 활용 데이터

법제처 국가법령정보센터에서 받은 **도로교통법 PDF(73쪽)** 한 개를 사용했습니다. 조문 235개를 검색 단위 944개(청크)로 나누어 벡터DB에 저장했습니다.

1.1 데이터 개요

항목	내용
문서명	도로교통법
출처	법제처 국가법령정보센터 (PDF 내려받기)
판	[시행 2018. 4. 25.] [법률 제15530호, 2018. 3. 27., 일부개정]
분량	73쪽, 220KB, 텍스트 PDF (스캔본 아님, OCR 불필요)
구성	본문 제1조 ~ 제166조 + 부칙 제1조 ~ 제4조, 장(章) 단위로 구분
형식	조문마다 '제N조(제목)'으로 시작하고, 그 아래 ①②③ 항, 1. 2. 3. 호, 가. 나. 목이 이어짐

1.2 이 데이터를 고른 이유

- **구조가 분명함:** 모든 조문이 '제N조(제목)'으로 시작하므로, 의미 단위로 자르는 방법(청킹)을 시험하고 검증하기 좋습니다.
- **정답 확인이 쉬움:** 검색 결과가 맞는지 조문 번호로 바로 채점할 수 있습니다.
- **말투 차이가 큼:** 사람은 '휴대폰', '헬멧'이라고 묻지만 법은 '휴대용 전화', '인명보호 장구'라고 씁니다. 단어가 달라도 뜻으로 찾는 의미 검색을 시험하기에 알맞습니다.
- **누구나 아는 주제:** 음주운전, 어린이 보호구역처럼 질문을 만들기 쉽고, 결과를 직관적으로 판단할 수 있습니다.

1.3 데이터 통계 (처리 단계별)

단계	개수	설명
PDF 쪽	73	원본
추출한 줄	2,979	머리말·꼬리말 147줄 제거 후
조문	235	'제N조(제목)' 기준으로 묶음. 이중 193개는 길어서 여러 청크로 나뉨
청크(검색 단위)	944	180자 이하로 나눈 조각
청크 길이	평균 153자	최소 30자, 중앙값 159자, 최대 212자 (이름표 포함)
벡터	944 × 768	청크 1개당 숫자 768개

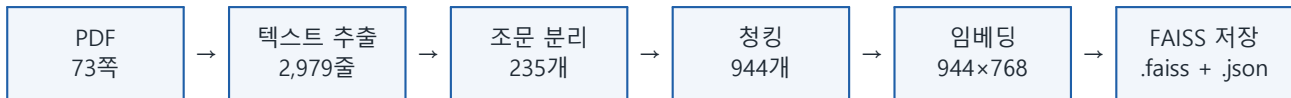
1.4 데이터에서 발견한 특징 (전처리할 때 주의할 점)

특징	예시	처리 방법
모든 쪽에 같은 머리말·꼬리말	'도로교통법', '법제처 1 국가법령정보센터'	정규식으로 찾아서 삭제
같은 조문이 두 번 실림	제44조 등이 현행판과 시행 예정판([시행일 : 2019. 6. 13.])으로 중복	이번에는 그대로 둠 → 한계점으로 기록(4.4)
조문 번호가 다시 시작됨	본문 제166조 뒤 부칙에 제1조~제4조가 또 나옴	쪽번호를 함께 저장해 구분
단어가 줄바꿈에서 끊김	'운 전하여서는', '한 다'	이번에는 그대로 둠 → 한계점으로 기록
옛날 판	음주 기준 0.05% (현행 법은 0.03%)	과제용 실습이므로 받은 판 그대로 사용

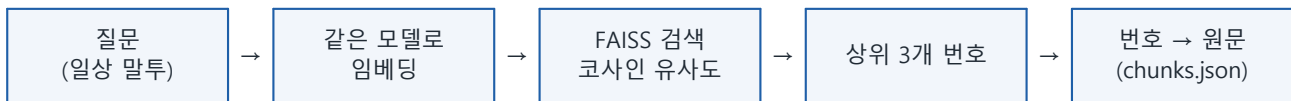
2. 프로세스 설계

전체 과정은 만들기(한 번만 실행)와 찾기(질문할 때마다 실행) 두 부분으로 나누었습니다. 만들기 결과를 파일로 저장해 두면, 검색할 때는 PDF를 다시 읽거나 임베딩을 다시 계산할 필요가 없습니다.

2.1 전체 흐름 ① 만들기 (build_index.py)



2.1 전체 흐름 ② 찾기 (search.py)



2.2 단계별 설계

단계	입력 → 출력	도구	설계 이유
① 텍스트 추출	PDF → (쪽번호, 줄) 목록	pdfplumber	한글 PDF에서 줄 단위로 안정적으로 추출됨. 쪽번호는 결과 검증용으로 함께 저장
② 조문 분리	줄 → 조문	정규식	법률은 조문 하나가 하나의 주제. 글자 수로 자르면 두 조문이 한 조각에 섞임
③ 청킹	조문 → 180자 이하 청크	직접 구현	임베딩 모델이 128토큰(한글 약 210자)까지만 읽기 때문 (2.3 참고)
④ 임베딩	청크 → 768차원 벡터	sentence-transformers	한국어 문장 유사도 학습 모델 사용. 길이를 1로 정규화
⑤ 저장	벡터 → 인덱스 파일	faiss-cpu	서버 없이 파일 하나로 저장·로드 가능
⑥ 검색	질문 → 상위 k개 청크	faiss-cpu	질문도 같은 모델·같은 정규화로 변환해야 같은 공간에서 비교 가능

2.3 핵심 설계 결정

결정	선택	다른 선택지	이유
자르는 기준	조문 → 줄 단위	고정 글자 수(예: 500자)	문장과 조문 경계를 지켜 뜻이 섞이지 않게 함
청크 크기	180자	800자 (처음 시도)	모델 최대 입력 128토큰 210자. 800자면 뒤 약 590자가 버려짐
청크 앞 이름표	[제N조(제목)] 붙임	안 붙임	두 번째 이후 조각도 어느 조문의 내용인지 모델이 알 수 있음
임베딩 모델	jhgan/ko-sroberta-multitask	OpenAI 등 유료 API	무료, 로컬 실행, 한국어 문장 유사도(KorSTS) 학습
유사도	코사인 (정규화 + 내적)	L2 거리	문장 길이와 상관없이 방향(뜻)만 비교
인덱스	IndexFlatIP (전수 비교)	IVF, HNSW (근사)	944개는 적어서 전부 비교해도 47ms. 근사 방식은 정확도만 낮음

2.4 평가 설계

검색 품질을 숫자로 판단하기 위해, **결과를 보기 전에** 질문 10개와 각 질문의 정답 조문을 먼저 정했습니다.

질문 유형	개수	예시	확인하려는 것
다른 단어로 묻기	3	'휴대폰' → 휴대용 전화, '헬멧' → 인명보호 장구, '갓길로 달려도'	단어가 달라도 뜻으로 찾는가
숫자·정확한 기준 묻기	1	혈중알코올농도 기준은?	숫자 정보를 찾는가 (어려운 문제)
일반 질문	6	주차 금지 장소, 면허 취소, 사고 후 조치 등	기본 검색 성능

채점 지표는 두 가지입니다.

- **Hit@1:** 정답 조문이 1위로 나온 질문의 비율
- **Hit@3:** 정답 조문이 3위 안에 든 질문의 비율. 검색 결과 여러 개를 AI에 함께 넘기는 RAG 방식에서 중요
- 조문 번호가 같으면 정답으로 채점했습니다. 같은 조문 안의 다른 문단이 나와도 맞은 것으로 보았습니다(4.3에서 이 기준의 한계를 다룸).

3. 구현

3.1 실행 환경

항목	버전 / 사양
운영체제	Windows 11 Pro
언어	Python 3.13
벡터DB	faiss-cpu 1.15.1
임베딩	sentence-transformers 6.1.0 + jhgan/ko-sroberta-multitask (약 420MB)
PDF 추출	pdfplumber 0.11.10
연산 장치	CPU만 사용 (GPU 없음)

3.2 파일 구성

파일	역할
build_index.py	PDF 읽기 → 조문 분리 → 청킹 → 임베딩 → FAISS 저장 (만들기)
search.py	저장된 인덱스를 불러 질문을 검색하는 Searcher 클래스 + 명령줄 실행
run_experiments.py	질문 10개를 검색하고 Hit@1, Hit@3을 계산해 results.md로 저장
index/road_traffic.faiss	벡터 944개가 든 FAISS 인덱스 (2.8MB)
index/chunks.json	각 벡터의 원문, 조문 번호, 제목, 쪽번호 (벡터와 같은 순서)
results.md	질문별 상위 3개 검색 결과 원문

3.3 핵심 코드

```

NOISE = re.compile(r"^(도로교통법|법제처\s+\d+\s+국가법령정보센터)$") # 머리말·꼬리말
ARTICLE_START = re.compile(r"^(제\d+조(?:의\d+)?\(((^)+)\))") # 제N조(제목), 제N조의M(제목)

for page_no, line in lines:
    m = ARTICLE_START.match(line)
    if m: # 새 조문 시작 → 이전 조문 저장
        if cur: articles.append(cur)
        cur = {"article": m.group(1), "title": m.group(2), "page": page_no, "text": line}
    elif cur: # 아니면 지금 조문 뒤에 이어 붙임
        cur["text"] += "\n" + line

```

[코드 1] 텍스트 정리와 조문 분리 (build_index.py)

```

MAX_CHARS = 180 # 모델 최대 입력 128토큰(한글 약 210자) 안에 들어가도록

header = f"[a['article']][a['title']]"
for line in a["text"].splitlines():
    if buf and len(buf) + len(line) > MAX_CHARS: # 이 줄을 넣으면 넘친다 → 조각 완성
        parts.append(buf); buf = ""
    buf += line + "\n"
...
text = p.strip() if i == 0 else header + p.strip() # 두 번째 조각부터 이름표 붙이기

```

[코드 2] 줄 단위 청킹과 조문 이름표 (build_index.py)

```

model = SentenceTransformer("jhgan/ko-sroberta-multitask")
emb = model.encode(texts, batch_size=32, normalize_embeddings=True).astype("float32")

index = faiss.IndexFlatIP(emb.shape[1])    # 768차원, 내적(= 정규화 후 코사인 유사도)
index.add(emb)
faiss.write_index(index, "index/road_traffic.faiss")
json.dump(chunks, open("index/chunks.json", "w", encoding="utf-8"), ensure_ascii=False)

```

[코드 3] 임베딩과 FAISS 인덱스 저장 (build_index.py)

```

def search(self, query, k=3):
    q = self.model.encode([query], normalize_embeddings=True).astype("float32")
    scores, ids = self.index.search(q, k)    # 가장 가까운 k개의 번호와 점수
    return [{"score": float(s)}            # 번호 → chunks.json의 원문
            for s, i in zip(scores[0], ids[0])]

```

[코드 4] 검색 (search.py)

3.4 실행 방법과 소요 시간

```

pip install -r requirements.txt
python build_index.py          # 인덱스 만들기 (한 번만)
python search.py "음주운전 처벌" -k 5 # 직접 검색
python run_experiments.py      # 질문 10개 채점 → results.md

```

작업	소요 시간 (CPU)	비고
PDF 텍스트 추출 (73쪽)	약 5.4초	
청크 944개 임베딩	약 100초	가장 오래 걸림. 그래서 결과를 파일로 저장해 둠
인덱스·모델 불러오기	약 3.9초	검색 프로그램 시작 시 한 번
질문 1개 검색	약 47ms	질문 임베딩 + FAISS 검색 포함

3.5 구현 중 겪은 문제와 해결

문제	원인	해결
처음 만든 800자 청크는 뒷부분이 임베딩에 반영되지 않음	모델 최대 입력이 128토큰. 800자는 약 420~500토큰	청크를 180자로 줄임 (296개 → 944개). 944개 중 942개가 128토큰 이내
잘린 조각만 보면 어느 조문인지 알 수 없음	'③ 측정 결과에 불복하는...'처럼 조문명이 없음	모든 조각 앞에 [제N조(제목)] 이름표를 붙임
장(章) 제목이 앞 조문 끝에 붙음	'제2장 보행자의 통행방법' 줄이 조문 본문으로 처리됨	장 제목 정규식을 따로 만들어 본문에서 제외
터미널에 한글이 깨져 출력됨	Windows 콘솔 인코딩(cp949)	PYTHONIOENCODING=utf-8로 실행. 파일은 UTF-8로 저장

4. 결과 및 한계점

질문 10개 중 8개는 정답 조문이 1위(Hit@1 = 80%), 10개 모두 3위 안(Hit@3 = 100%)에 나왔습니다. 단어가 달라도 뜻으로 찾는 것은 잘 됐지만, 비슷한 조문들 사이의 순위는 점수 차이가 0.01 미만으로 아슬아슬했습니다.

4.1 질문별 검색 결과

#	질문	정답	1위 결과	1위 점수	정답 순위
1	음주운전으로 보는 혈중알코올농도 기준은?	제44조	제93조(운전면허의 취소·정지)	0.612	3
2	술 마시고 운전하면 어떤 처벌을 받나요?	제148조의2	제148조의2(벌칙)	0.684	1
3	어린이 보호구역에서 지켜야 할 것	제12조	제12조(어린이 보호구역의 지정 및 관리)	0.754	1
4	운전 중에 휴대폰 사용해도 되나요?	제49조	제49조(모든 운전자의 준수사항 등)	0.520	1
5	고속도로에서 갓길로 달려도 되나요?	제60조	제60조(갓길 통행금지 등)	0.566	1
6	교통사고를 내면 운전자가 해야 할 조치	제54조	제29조(긴급자동차의 우선 통행)	0.728	3
7	운전면허가 취소되는 경우	제93조	제93조(운전면허의 취소·정지)	0.759	1
8	자전거 탈 때 헬멧을 써야 하나요?	제50조	제50조(특정 운전자의 준수사항)	0.552	1
9	주차가 금지된 장소는 어디인가?	제33조	제33조(주차금지의 장소)	0.629	1
10	긴급자동차에게 길을 양보해야 하는 의무	제29조	제29조(긴급자동차의 우선 통행)	0.774	1

점수 = 코사인 유사도 (1에 가까울수록 비슷함). 질문별 상위 3개 원문은 저장소의 results.md에 있음.

지표	결과
Hit@1	8 / 10 (80%)
Hit@3	10 / 10 (100%)
1위 점수 범위	0.520 ~ 0.774 (평균 0.658)
관련 없는 질문의 1위 점수 (참고)	'오늘 저녁 메뉴 추천' → 0.19

4.2 잘 된 사례: 단어가 달라도 뜻으로 찾음

질문	1위로 찾은 원문 (일부)	점수
운전 중에 휴대폰 사용해도 되나요?	[제49조] 10. 운전자는 자동차등 또는 노면전차의 운전 중에는 휴대용 전화 (자동차용 전화를 포함한다)를 사용하지 아니할 것. 다만, ... 정지하고 있는 경우 ... 그러하지 아니하다.	0.520
고속도로에서 갓길로 달려도 되나요?	제60조(갓길 통행금지 등)	0.566

질문에는 '휴대용 전화'라는 단어가 없는데도 정확한 문단을 찾았고, 예외 조건(정차 중, 긴급자동차)까지 함께 나왔습니다. 단어가 겹쳐야 찾는 키워드 검색으로는 찾기 어려운 경우입니다.

4.3 실패 사례 분석

질문	1~3위 (점수)	원인 분석
Q1. 혈중알코올농도 기준은?	제93조 (0.6125) 제148조의2 (0.6112) 제44조 (0.6106)	제93조(면허 취소)와 제148조의2(벌칙)도 '제44조 위반', '술에 취한 상태'를 반복해서 뜻이 비슷하게 잡힘. 정답 문장 '혈중알코올농도가 0.05퍼센트 이상'은 3위. 의미 검색은 숫자 일치를 잘 반영하지 못함.
Q6. 교통사고 후 운전자 조치	제29조 (0.7284) 제49조 (0.7229) 제54조 (0.7199)	'운전자는 ~하여야 한다'는 의무 문장 구조가 비슷한 조문이 위로 올라옴. 정답과 1위의 차이는 0.009점.
Q8. 자전거 헬멧 (조문은 맞음)	제50조 ⑧⑨항 (0.552) 제15조의2 (0.549) 제50조 ③④항 (0.509)	조문 번호로는 1위 정답이지만, 1위 문단은 약물·전조등 내용이었음. 헬멧 문장('인명보호 장구')이 든 청크는 3위였고, 그 문장도 청크 경계에서 '인명보호 / 장구를 착용하여야'로 잘려 있었음.

세 경우 모두 1~3위 점수 차이가 0.05 미만이었습니다. 법조문끼리는 문체와 단어가 비슷해서, 임베딩만으로는 순위가 쉽게 뒤바뀝니다. 또 Q8은 **조문 단위로 채점하면 실제보다 결과가 좋아 보일 수** 있다는 점을 보여 줍니다.

4.4 한계점

한계	영향	원인
숫자·정확한 용어에 약함	Q1처럼 기준값을 묻는 질문에서 순위가 밀림	임베딩은 문장의 전체 뜻만 비교
비슷한 조문 간 구별이 약함	1~3위 점수 차이 0.01 미만	법조문의 문체·단어가 비슷함
청크 경계에서 문장이 잘림	Q8처럼 핵심 문장이 두 청크로 나뉨	줄 단위로 180자에 맞춰 자름. 줄이 문장 경계는 아님
중복 청크	같은 내용이 2번 나와 상위 3개 자리를 차지할 수 있음	PDF에 현행판·시행 예정판이 함께 실림
줄바꿈으로 끊긴 단어	'운 전하여서는' 같은 표현이 임베딩 품질을 낮춤	PDF 줄바꿈이 그대로 추출됨
평가 규모가 작음	질문 10개로는 수치의 신뢰도가 낮음 (1문제 = 10%p)	질문과 정답을 손으로 만들
2018년 판 법률	현행 법과 다른 답 (음주 기준 0.05% → 현행 0.03%)	받은 PDF가 2018년 판

4.5 개선 방향

개선 방법	해결하는 한계	방법
하이브리드 검색	숫자·용어에 약함	BM25 키워드 검색 점수와 벡터 검색 점수를 합산
리랭커 추가	비슷한 조문 구별	상위 20개를 Cross-Encoder로 다시 정렬
문장·항 단위 청킹 + 겹치기	청크 경계에서 잘림	①②③ 항 기준으로 자르고, 앞뒤 청크를 조금씩 겹치게(overlap) 함
시행일 판별	중복 청크	[시행일 : ...] 표시를 읽어 한 판만 남김
줄바꿈 복원	끊긴 단어	한글 사이 줄바꿈을 붙여서 추출
평가셋 확대	평가 규모	질문 50개 이상, 문단 단위 정답으로 채점

4.6 결론

FAISS와 무료 한국어 임베딩 모델만으로도, 법률 문서에서 일상 말투 질문의 정답 조문을 10개 중 8개는 1위로, 10개 모두 3위 안에서 찾았습니다. 가장 크게 배운 점은 **청크 크기를 모델의 최대 입력 길이에 맞춰야 한다**는 것입니다. 처음 800자로 잘랐을 때는 대부분의 내용이 임베딩에 반영되지 않았습니다. 숫자를 묻는 질문과 비슷한 조문 사이의 순위는 아직 약합니다. 다음 단계로 키워드 검색을 섞은 하이브리드 검색과, 항 단위 청킹을 적용해 볼 계획입니다.